

INT234: PREDICTIVE ANALYTICS
PROJECT REPORT
(Project Semester October 2025 -January 2026)

House Price Prediction And Classification

Submitted by

Sudeep Kumar Reddy Eaga

Registration No: 12318529

Section: KM003

Course Code: INT 234

Under the Guidance of

Dr. Mrinalini Rana

22138

Discipline of CSE/IT

Lovely School of Computer Science

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Sudeep Kumar Reddy Eaga bearing Registration no. 12318529 has completed INT 234 project titled, “**House Price Prediction And Classification**” under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor

Designation of the Supervisor

School of Computer Science.

Lovely Professional University

Phagwara, Punjab.

Date:

DECLARATION

I, Sudeep Kumar Reddy Eaga, student of B-tech Computer Science and Engineering under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 14-11-2025

Signature Sudeep Kumar Reddy Eaga

Registration No. 12318529

1. Introduction

In recent years, the real estate sector has increasingly adopted data-driven approaches to support informed decision-making and accurate property valuation. Predictive analytics, combined with machine learning techniques, plays a crucial role in analyzing large volumes of housing data to identify market trends and estimate property prices effectively.

This project focuses on **house price prediction and classification** using supervised machine learning techniques. By analyzing key property-related attributes such as location, area (square footage), number of bedrooms and bathrooms, age of the property, amenities, and neighbourhood characteristics, the model aims to predict house prices and classify properties into different price ranges. Such predictions can assist buyers, sellers, and real estate professionals in understanding market value and making data-driven investment decisions.

The project involves **exploratory data analysis (EDA)** to uncover underlying patterns and relationships within the dataset, followed by data preprocessing steps including handling missing values, encoding categorical variables, and feature scaling. Multiple machine learning algorithms are implemented, including **Linear Regression, K-Nearest Neighbours (KNN), Support Vector Machine (SVM), Decision Tree, and Random Forest**, to perform both regression and classification tasks.

Model performance is evaluated using appropriate metrics such as **R^2 score, Mean Absolute Error (MAE), Root Mean Squared Error (RMSE)** for price prediction, and **accuracy, confusion matrix, and classification report** for price range classification. Advanced visualizations are used to interpret model behaviour and compare results across algorithms. The final outcome includes a **comparative analysis of models** to identify the best-performing approach for accurate house price prediction and classification.

2. Source of dataset

The dataset used in this project is sourced from **Kaggle**, a well-known open platform for data science and machine learning resources. The dataset is titled “**House Sales in King County, USA**” and is publicly available at:

 <https://www.kaggle.com/datasets/harlfoxem/housesalesprediction>

This dataset contains information related to residential house sales and property characteristics, including attributes such as location, number of bedrooms and bathrooms, living area (square footage), lot size, number of floors, waterfront view, condition, grade, year built, and sale price. These features provide valuable insights into factors that influence housing prices.

The dataset is well-suited for **supervised machine learning tasks**, particularly **regression and classification problems**, as it includes both input features and a target variable (house sale price). In this project, the sale price is used to predict exact property values and to classify houses into different price ranges for comparative analysis.

The data was downloaded in **CSV format** from Kaggle and used solely for **academic and educational purposes**. Prior to analysis, the dataset underwent comprehensive preprocessing steps, including handling missing values, encoding categorical variables, normalization, and feature scaling, to ensure compatibility with machine learning models. These preprocessing steps help improve model performance, reliability, and interpretability during training and evaluation.

3. EDA process

3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is a crucial step in understanding the structure, quality, and underlying patterns of the dataset before applying machine learning models. In this project, EDA was conducted to gain insights into house sale data, identify relationships between features, and detect potential issues such as missing values, outliers, and data imbalance.

3.1 Data Inspection

The dataset was initially examined by viewing the first few records using functions such as `head()` and analyzing the dataset structure using `info()`. This step helped in understanding the number of observations, feature names, data types, and the distinction between numerical and categorical variables such as price, bedrooms, bathrooms, and location-based attributes.

3.2 Statistical Summary

Descriptive statistics were generated using the `describe()` function to analyze the central tendency, dispersion, and range of numerical attributes such as house price, living area (square footage), lot size, number of bedrooms, and bathrooms. This provided a high-level overview of data distribution, variability, and the presence of potential outliers.

3.3 Handling Categorical Variables

During EDA, categorical features such as location, waterfront view, condition, and grade were identified. These variables were transformed into numerical representations using encoding techniques to ensure compatibility with machine learning algorithms and to preserve meaningful information for model training.

3.4 Feature Scaling and Normalization

To ensure uniform contribution of all numerical features, scaling techniques were applied. Min-Max normalization was used during preprocessing, and standardization was applied during model training to improve the performance of distance-based algorithms like K-Nearest Neighbors (KNN) and margin-based models such as Support Vector Machine (SVM).

3.5 Target Variable Analysis

A target variable named **Price_Class** was created by categorizing house prices into distinct price ranges (such as low and high). This transformation enabled the conversion of the problem into a classification task. The

distribution of the target variable was analyzed using visualizations to assess class balance and identify any imbalance that could impact model performance.

Overall, the EDA process established a strong foundation for feature selection, preprocessing, and model development. It ensured that the dataset was clean, well-structured, and suitable for both house price prediction and classification using supervised machine learning techniques.

Screen Shots Of EDA:

1> Importing the libraries and Loading the Dataset:

[2]:

```
import pandas as pd
import numpy as np
import seaborn as sb
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.svm import SVC
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
```

[3]:

```
data = pd.read_csv("D:\\Downloads\\kc_house_data.csv")
data.head()
```

[3]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_renovated
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000	1.0	0	0	...	7	1050	910	1965	
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080	1.0	0	0	...	8	1680	0	1987	

5 rows × 21 columns

2> Using info() and describe() functions to examine dataset:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
 #   Column              Non-Null Count  Dtype  
---  --
 0   id                  21613 non-null  int64  
 1   date                21613 non-null  object  
 2   price               21613 non-null  float64 
 3   bedrooms            21613 non-null  int64  
 4   bathrooms           21613 non-null  float64 
 5   sqft_living         21613 non-null  int64  
 6   sqft_lot            21613 non-null  int64  
 7   floors              21613 non-null  float64 
 8   waterfront          21613 non-null  int64  
 9   view                21613 non-null  int64  
10   condition           21613 non-null  int64  
11   grade              21613 non-null  int64  
12   sqft_above          21613 non-null  int64  
13   sqft_basement       21613 non-null  int64  
14   yr_built            21613 non-null  int64  
15   yr_renovated        21613 non-null  int64  
16   zipcode             21613 non-null  int64  
17   lat                 21613 non-null  float64 
18   long                21613 non-null  float64 
19   sqft_living15       21613 non-null  int64  
20   sqft_lot15          21613 non-null  int64  
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
None
```

	id	price	bedrooms	bathrooms	sqft_living
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897
min	1.000102e+06	7.500000e+04	0.000000	0.000000	200.000000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000

	sqft_lot	floors	waterfront	view	condition
count	2.161300e+04	21613.000000	21613.000000	21613.000000	21613.000000
mean	1.510697e+04	1.494309	0.007542	0.234303	3.400430
std	4.142051e+04	0.539989	0.086517	0.766318	0.650743
min	5.200000e+02	1.000000	0.000000	0.000000	1.000000
25%	5.040000e+03	1.000000	0.000000	0.000000	3.000000
50%	7.618000e+03	1.500000	0.000000	0.000000	3.000000
75%	1.068800e+04	2.000000	0.000000	0.000000	4.000000
max	1.651350e+06	3.500000	1.000000	4.000000	5.000000

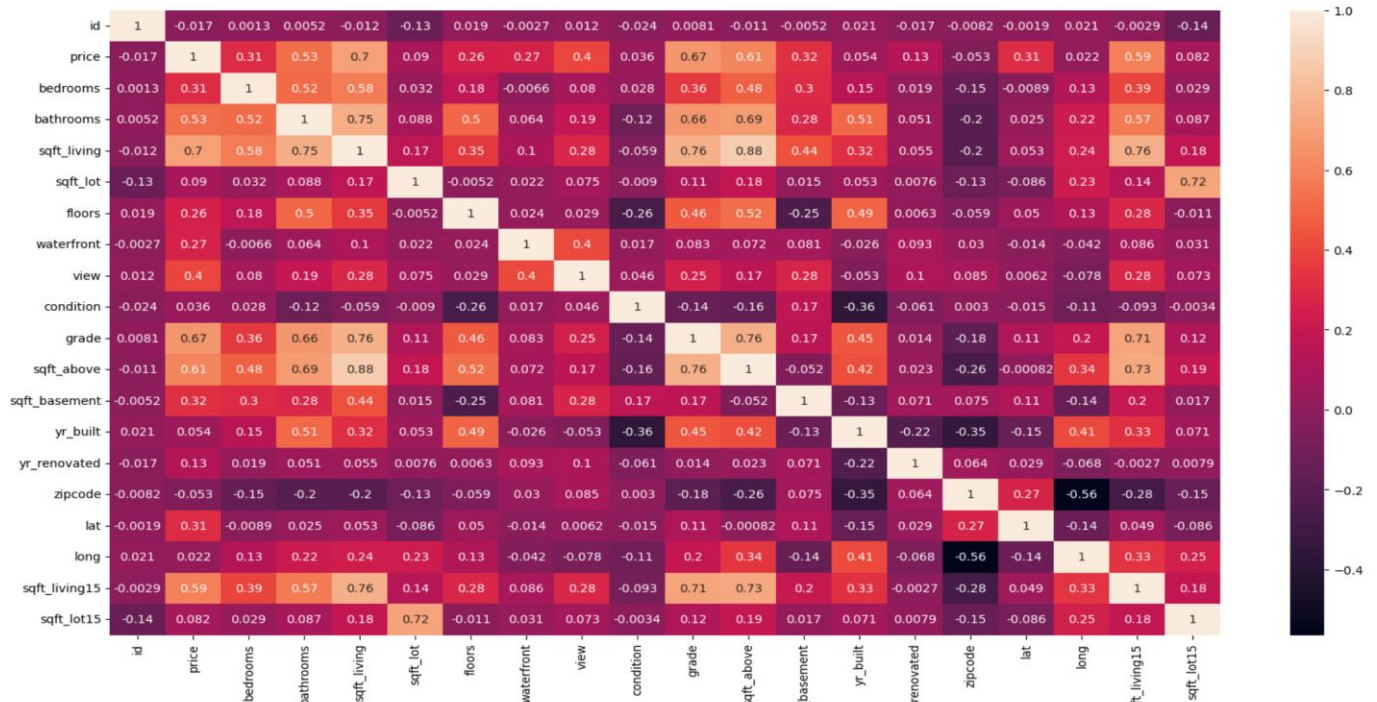
	grade	sqft_above	sqft_basement	yr_built	yr_renovated
count	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	7.656873	1788.300691	291.500045	1971.005136	84.402258
std	1.175459	828.000078	442.375043	29.373411	461.679240
min	1.000000	200.000000	0.000000	1900.000000	0.000000
25%	7.000000	1190.000000	0.000000	1951.000000	0.000000
50%	7.000000	1500.000000	0.000000	1975.000000	0.000000
75%	8.000000	2210.000000	560.000000	1997.000000	0.000000
max	13.000000	9410.000000	4820.000000	2015.000000	2015.000000

	zipcode	lat	long	sqft_living15	sqft_lot15
count	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	98077.939805	47.500053	-122.213896	1986.552402	12768.455652
std	53.505026	0.138564	0.140828	685.391304	27304.179631
min	98001.000000	47.155900	-122.519000	399.000000	651.000000
25%	98033.000000	47.471000	-122.328000	1450.000000	5100.000000
50%	98065.000000	47.571800	-122.230000	1840.000000	7620.000000
75%	98118.000000	47.678000	-122.125000	2360.000000	10983.000000
max	98199.000000	47.777600	-121.315000	6210.000000	871200.000000

3> Feature selection using heatmap:

```
[6]: data1 = data.select_dtypes(include = ['int', 'float'])
plt.figure(figsize = (20,10))
sb.heatmap(data1.corr(), annot=True)
```

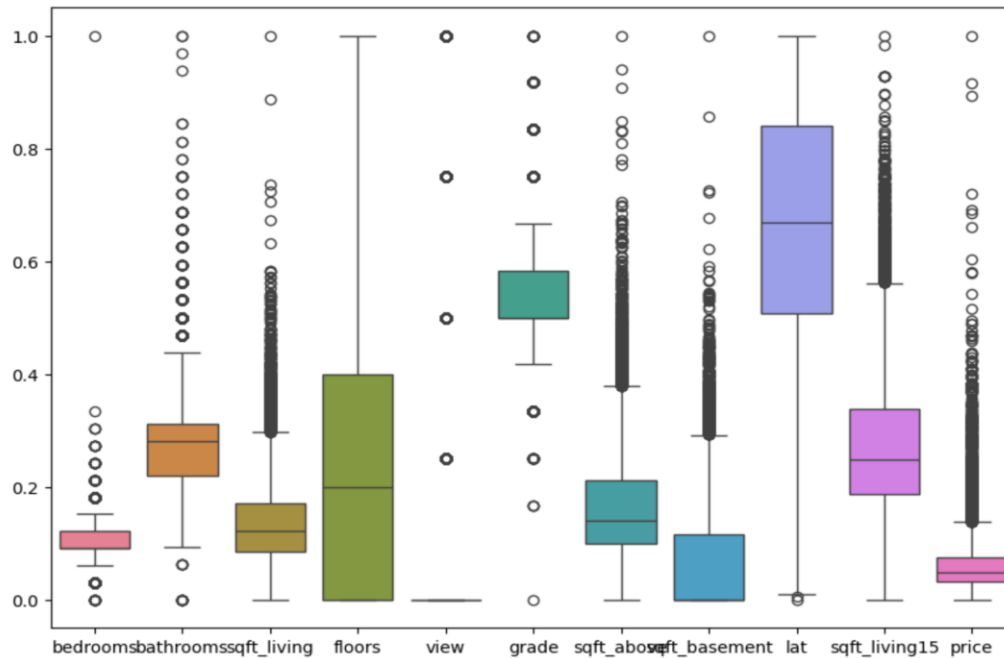
[6]: <Axes: >



4> Checking for outliers in selected columns:

```
[10]: plt.figure(figsize = (10,7))  
      sb.boxplot(data_norm)
```

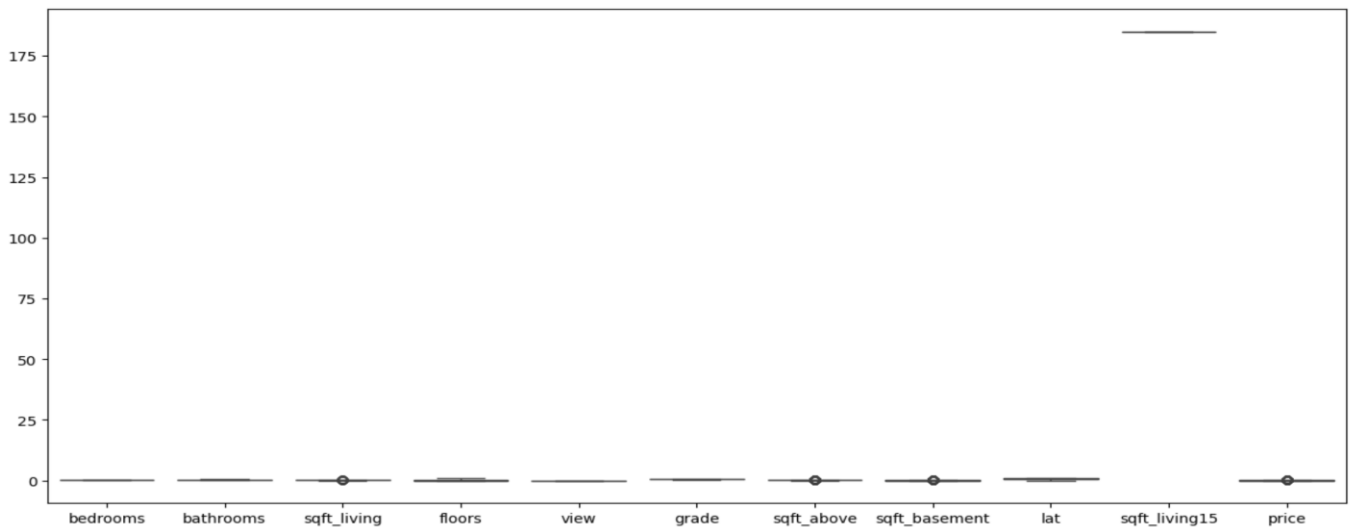
```
[10]: <Axes: >
```



5> Treating outliers using IQR method:

```
treatcols = ['sqft_living', 'sqft_above', 'sqft_basement', 'sqft_living15', 'price']  
for col in treatcols:  
    q1 = np.percentile(data[col], 25)  
    q3 = np.percentile(data[col], 75)  
    iqr = q3 - q1  
    lb = q1 - 1.5 * iqr  
    ub = q3 + 1.5 * iqr  
    data_norm[col] = data_norm[col].clip(lower=lb, upper=ub)  
plt.figure(figsize=(15,7))  
sb.boxplot(data_norm)
```

```
[13]: <Axes: >
```



4. Analysis on dataset

Introduction

The analysis phase focuses on extracting meaningful insights from the dataset through systematic examination of features and their relationship with the treatment outcome. This step bridges exploratory data analysis and machine learning modeling by identifying significant patterns, understanding data behavior, and preparing the dataset for effective classification. The analysis emphasizes both statistical and visual interpretation of patient and treatment-related attributes.

i. General Description

The dataset consists of patient-related information and drug treatment details, including numerical and categorical variables. Key attributes analyzed include:

- 1.sqft_living¹⁵
- 2.lat
- 3.sqft_basement
- 4.sqft_above
- 5.grade
- 6.view
- 7.floors
- 8.sqft_living
- 9.bathrooms
- 10.bedrooms.
- 11.Price Class

The outcome variable, **Price_Class**, represents whether a house falls into a specific price category (such as low or high value). The dataset was analyzed to understand the class distribution, feature relevance, and correlations among variables influencing house prices. This analysis ensured the suitability of the dataset for supervised classification models and helped identify key factors contributing to property value categorization.

ii. Specific Requirements, functions and formulas

1> Feature Scaling

To normalize the range of features and improve model performance, **Min-Max Scaling** and **Standard Scaling** were applied.

Min-Max Scaling Formula:

$$X_{\text{scaled}} = \frac{X_{\text{max}} - X_{\text{min}}}{X - X_{\text{min}}}$$

Standardization Formula:

$X_{\text{standardized}} = X_{\text{mean}} / \text{standard deviation}$

Screen shot of Feature Scaling:

```
[9]: #Normalizing data using minmax scaler
scaler = MinMaxScaler()
scaled_values = scaler.fit_transform(data[cols])
data_norm = pd.DataFrame(scaled_values, columns=cols, index=data.index)
print(data[cols].describe())
print(data_norm.describe())
```

	bedrooms	bathrooms	sqft_living	floors	view
count	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	3.370842	2.114757	2079.899736	1.494309	0.234303
std	0.930062	0.770163	918.440897	0.539989	0.766318
min	0.000000	0.000000	290.000000	1.000000	0.000000
25%	3.000000	1.750000	1427.000000	1.000000	0.000000
50%	3.000000	2.250000	1910.000000	1.500000	0.000000
75%	4.000000	2.500000	2550.000000	2.000000	0.000000
max	33.000000	8.000000	13540.000000	3.500000	4.000000

	grade	sqft_above	sqft_basement	lat	sqft_living15
count	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	7.656873	1788.390691	291.509045	47.560053	1986.552492
std	1.175459	828.090978	442.575043	0.138564	685.391304
min	1.000000	290.000000	0.000000	47.155900	399.000000
25%	7.000000	1190.000000	0.000000	47.471800	1490.000000
50%	7.000000	1560.000000	0.000000	47.571800	1840.000000
75%	8.000000	2210.000000	560.000000	47.678000	2360.000000
max	13.000000	9410.000000	4820.000000	47.777600	6210.000000

	price
count	2.161300e+04
mean	5.400881e+05
std	3.671272e+05
min	7.500000e+04
25%	3.219500e+05
50%	4.500000e+05
75%	6.450000e+05
max	7.700000e+06

	bedrooms	bathrooms	sqft_living	floors	view
count	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	0.102147	0.264345	0.135087	0.197724	0.058576
std	0.028184	0.096270	0.069316	0.215996	0.191579
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.090909	0.218750	0.085811	0.000000	0.000000
50%	0.090909	0.281250	0.122264	0.200000	0.000000
75%	0.121212	0.312500	0.170566	0.400000	0.000000
max	1.000000	1.000000	1.000000	1.000000	1.000000

	grade	sqft_above	sqft_basement	lat	sqft_living15
count	21613.000000	21613.000000	21613.000000	21613.000000	21613.000000
mean	0.554739	0.164297	0.060479	0.650076	0.273198
std	0.097955	0.090799	0.091821	0.222879	0.117947
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.500000	0.098684	0.000000	0.506836	0.187747
50%	0.500000	0.139254	0.000000	0.668972	0.247978
75%	0.583333	0.210526	0.116183	0.839794	0.337463
max	1.000000	1.000000	1.000000	1.000000	1.000000

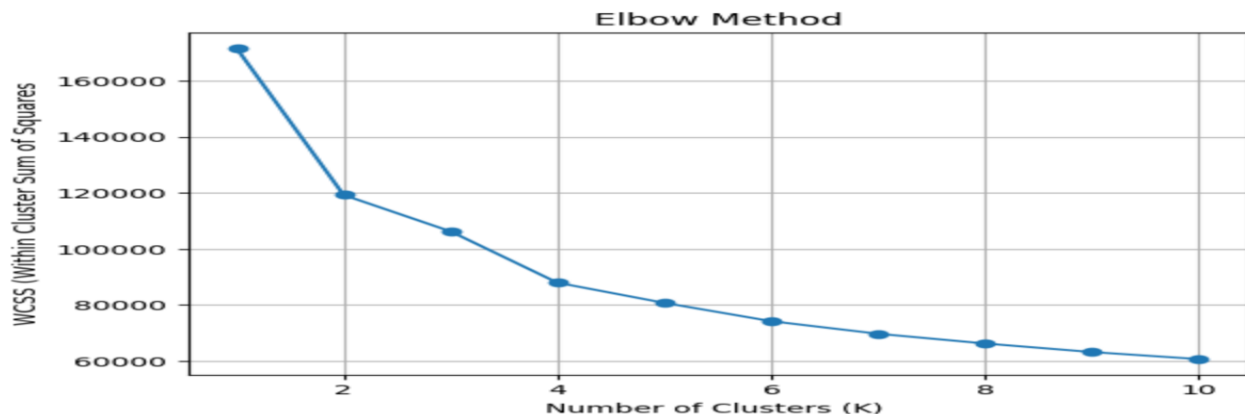
3>Target Variable Creation using k-means cluster

Applied WCSS method to create target variable and used elbow graph for its visualization and find out the number of cluster need to be used for best results.

```
#Finding Best K val using WCSS
wcss = []

for k in range(1, 11):
    km1 = KMeans(n_clusters=k, init='k-means++', random_state=0)
    km1.fit(data_norm)
    wcss.append(km1.inertia_)

plt.plot(range(1, 11), wcss, marker = 'o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K) ")
plt.ylabel("WCSS (Within Cluster Sum of Squares)")
plt.grid(True)
plt.show()
print(wcss)
```



```
#Creating Clusters
```

```
k = 4
km = KMeans(n_clusters = k, random_state = 0)
km.fit(data_norm)
data_norm['Price Class'] = km.labels_
print(data_norm)
```

	bedrooms	bathrooms	sqft_living	floors	view	grade	sqft_above	\
0	-0.353898	-1.528377	-1.063307	-0.861076	0.0	-0.492262	-0.713171	
1	-0.353898	0.413555	1.136028	0.992769	0.0	-0.492262	0.889759	
2	-1.653191	-1.528377	-1.712031	-0.861076	0.0	-1.712429	-1.377010	
3	0.945394	1.578714	0.170852	-0.861076	0.0	-0.492262	-0.923656	
4	-0.353898	0.025168	-0.272179	-0.861076	0.0	0.727905	0.096390	
...	...	...	...	...	...	...	...	
21608	-0.353898	0.801941	-0.509517	2.846614	0.0	0.727905	-0.146478	
21609	0.945394	0.801941	0.724641	0.992769	0.0	0.727905	1.116436	
21610	-1.653191	-1.916764	-1.316468	0.992769	0.0	-0.492262	-0.972230	
21611	-0.353898	0.801941	-0.398760	0.992769	0.0	0.727905	-0.033140	
21612	-1.653191	-1.916764	-1.316468	0.992769	0.0	-0.492262	-0.972230	

	sqft_basement	lat	sqft_living15	price	Price Class
0	-0.643697	-0.314572	-0.937149	-1.224047	0
1	0.468273	1.150155	-0.267394	0.509671	2
2	-0.643697	1.268143	1.703600	-1.453857	0
3	1.886036	-0.247549	-0.898877	0.871662	1
4	-0.643697	0.422679	-0.056899	0.356099	2
...	...	...	...	...	...
21608	-0.643697	0.998656	-0.573568	-0.466608	2
21609	-0.643697	-0.318063	0.000508	-0.247219	2
21610	-0.643697	0.266292	-1.549497	-0.235696	0
21611	-0.643697	-0.151902	-0.803198	-0.247219	2
21612	-0.643697	0.264198	-1.549497	-0.658573	0

[17160 rows x 12 columns]

4>Train-Test Split

The dataset was split into training and testing sets to evaluate model generalization.

Function Used:

`train_test_split(test_size=0.3, random_state=131)`

5>Classification Algorithms for Price Class prediction

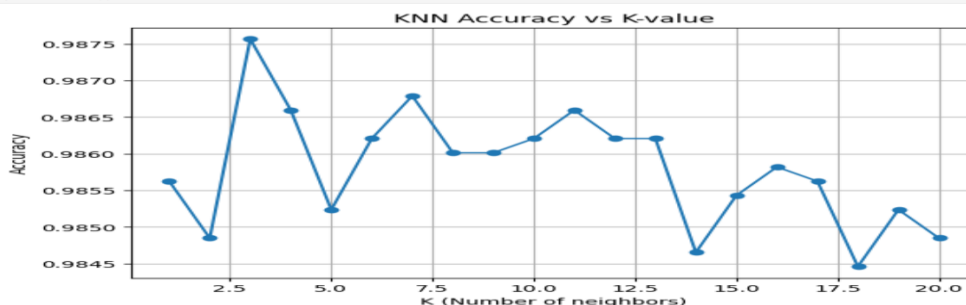
The following supervised learning algorithms were applied:

- K-Nearest Neighbors (KNN)

```
[28]: #Finding best K value for KNN model
k_values = range(1, 21)
accuracies = []

for k in k_values:
    knnmodel = KNeighborsClassifier(n_neighbors=k)
    knnmodel.fit(xc_train, yc_train)
    accuracies.append(accuracy_score(knnmodel.predict(xc_test), yc_test))

# Plot
plt.figure(figsize=(7,5))
plt.plot(k_values, accuracies, marker='o')
plt.xlabel("K (Number of neighbors)")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy vs K-value")
plt.grid(True)
plt.show()
```



```
[29]: #Train KNN model
kval = 3
knn = KNeighborsClassifier(n_neighbors=kval)
knn.fit(xc_train, yc_train)
```

```
[29]: KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

```
[30]: #Predict Using KNN Model
knnpred = knn.predict(xc_test)
```

```
[31]: #Evaluation of model
print("Accuracy Score: ",accuracy_score(yc_test, knnpred) * 100)
print("Classification Report: ",classification_report(yc_test, knnpred))
print("Confusion Matrix: ",confusion_matrix(yc_test, knnpred))
```

```
Accuracy Score: 98.75679875679876
Classification Report:
              precision    recall  f1-score   support

      0       0.98      0.99      0.99      1738
      1       0.99      0.99      0.99      1339
      2       0.98      0.99      0.99       721
      3       0.99      0.98      0.98      1350

   accuracy          0.99
  macro avg          0.99
 weighted avg          0.99

Confusion Matrix: [[1722    2    0   14]
 [   3 1325   11    0]
 [   0    6   711    4]
 [  24    0    0 1326]]
```

• Support Vector Machine (SVM)

```
[39]: #Training SVM model
svmmodel = SVC(kernel = 'linear')
svmmodel.fit(xc_train,yc_train)
```

```
[39]: SVC
SVC(kernel='linear')
```

```
[40]: #Predict Using SVM model
svmpred = svmmodel.predict(xc_test)
```

```
[41]: #Evaluation of SVM model
print("\nConfusion Matrix:\n", confusion_matrix(yc_test, svmpred))
print(f"\nAccuracy: {accuracy_score(yc_test, svmpred) * 100:.2f}%")
print("\n Classification Report: ",classification_report(yc_test, svmpred))
```

```
Confusion Matrix:
[[1721    0    0   17]
 [   7 1331    1    0]
 [   0   12   698   11]
 [  12    0    0 1338]]

Accuracy: 98.83%

Classification Report:
              precision    recall  f1-score   support

      0       0.99      0.99      0.99      1738
      1       0.99      0.99      0.99      1339
      2       1.00      0.97      0.98       721
      3       0.98      0.99      0.99      1350

   accuracy          0.99
  macro avg          0.99
 weighted avg          0.99
```

Naive Bayes

```
[32]: #Train Naive Bayes Model
NBmodel = GaussianNB()
NBmodel.fit(xc_train, yc_train)
```

```
[32]: GaussianNB
GaussianNB()
```

```
[33]: #Predict Using NaiveBayes Model
NBpred = NBmodel.predict(xc_test)
```

```
[34]: #Evaluation of model
print("Accuracy Score: ",accuracy_score(yc_test, NBpred) * 100)
print("Classification Report: ",classification_report(yc_test, NBpred))
print("Confusion Matrix: ",confusion_matrix(yc_test, NBpred))
```

Accuracy Score: 92.83216783216784

Classification Report:

		precision	recall	f1-score	support
--	--	-----------	--------	----------	---------

0	0.96	0.92	0.94	1738
1	0.91	0.93	0.92	1339
2	0.87	0.92	0.90	721
3	0.94	0.94	0.94	1350

accuracy		0.93	5148
macro avg	0.92	0.93	5148
weighted avg	0.93	0.93	5148

Confusion Matrix: [[1606 67 0 65]

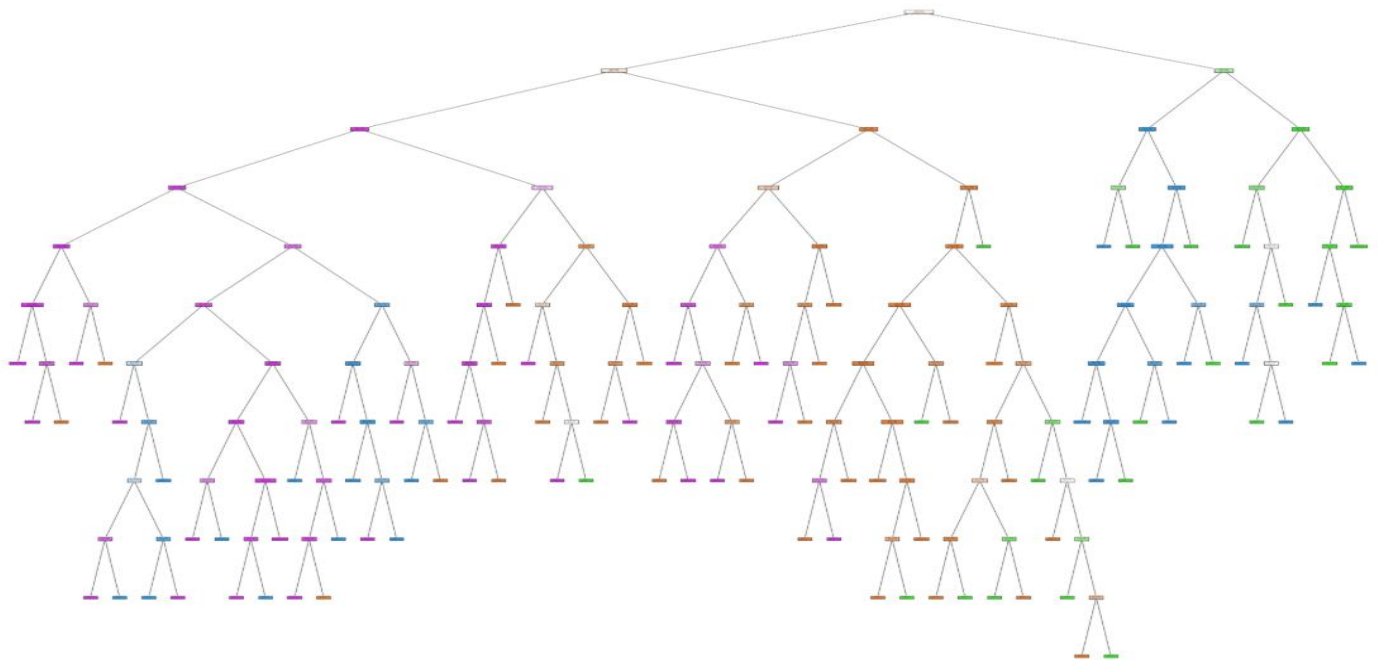
[0 1242 97 0]
[0 45 665 11]
[66 18 0 1266]]

Decision Tree

```
[35]: #Train Decision Tree Model
treemodel = DecisionTreeClassifier()
treemodel.fit(xc_train, yc_train)
```

```
[35]: DecisionTreeClassifier
DecisionTreeClassifier()
```

```
[36]: #Plot Tree
plt.figure(figsize = (50, 50))
plot_tree(treemodel, filled = True)
plt.show()
```



```
#Predict Using Decision Tree
treepred = treemodel.predict(xc_test)
```

```
#Evaluation of tree model
print("\nConfusion Matrix:\n", confusion_matrix(yc_test, treepred))
print(f"\nAccuracy: {accuracy_score(yc_test, treepred) * 100:.2f}%")
print("\n Classification Report: ",classification_report(yc_test, treepred))
```

```
Confusion Matrix:
[[1732    1     0     5]
 [   3 1332     4     0]
 [   2     5  707     7]
 [  16     0     2 1332]]
```

```
[[1732    1    0    5]
 [   3 1332    4    0]
 [   2    5 707    7]
 [  16    0    2 1332]]
```

Accuracy: 99.13%

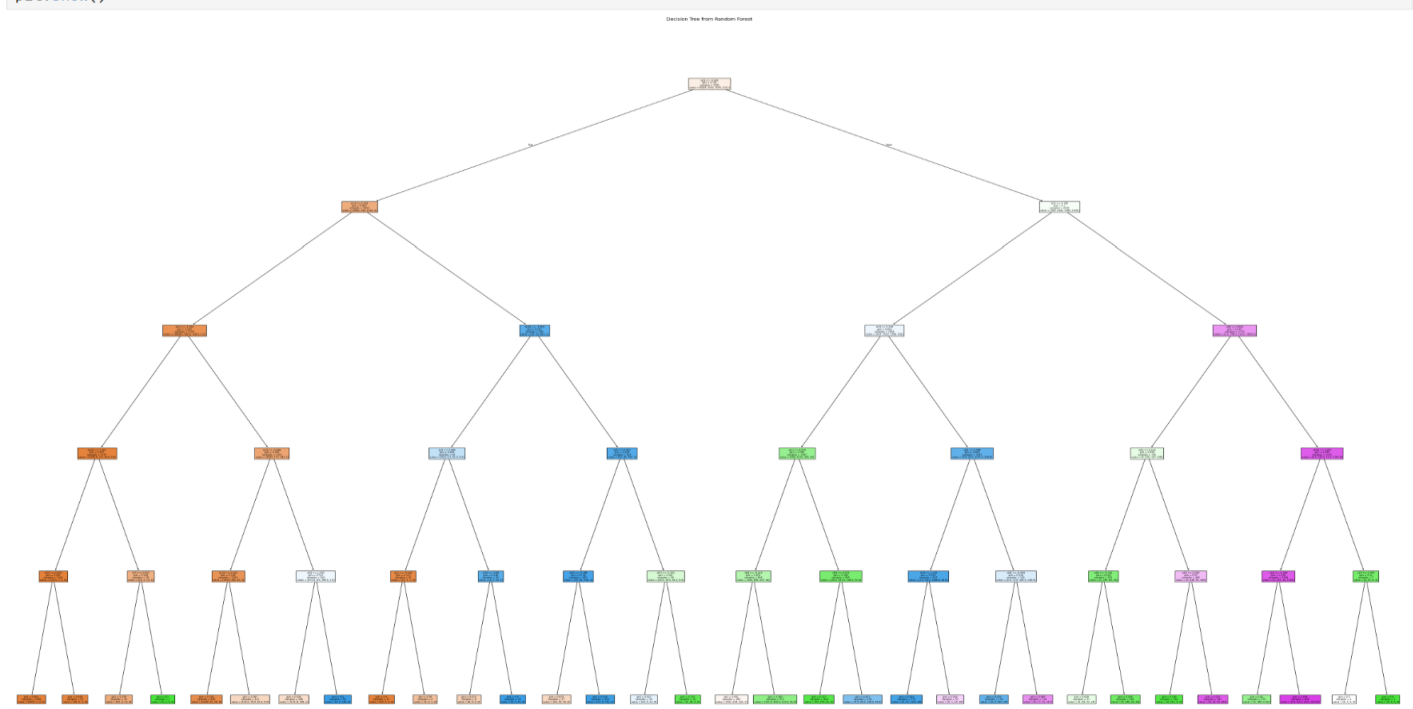
Classification Report:			precision	recall	f1-score	support
0	0.99	1.00	0.99	1738		
1	1.00	0.99	1.00	1339		
2	0.99	0.98	0.99	721		
3	0.99	0.99	0.99	1350		
accuracy			0.99	5148		
macro avg			0.99	0.99	0.99	5148
weighted avg			0.99	0.99	0.99	5148

- Random Forest

```
#Training Random Forest model
rf = RandomForestClassifier(n_estimators = 10, max_depth = 5)
rf.fit(xc_train, yc_train)
```

```
RandomForestClassifier(max_depth=5, n_estimators=10)
```

```
plt.figure(figsize = (60,40))
plot_tree(rf[0], filled = True)
plt.title("Decision Tree from Random Forest")
plt.show()
```



```
#Predict Using Random Forest
rfpred = rf.predict(xc_test)
```

```
#Evaluation of SVM model
print("\nConfusion Matrix:\n", confusion_matrix(yc_test, rfpred))
print(f"\nAccuracy: {accuracy_score(yc_test, rfpred) * 100:.2f}%")
print("\n Classification Report: ",classification_report(yc_test, rfpred))
```

```
Confusion Matrix:
[[1682   83   72    0]
 [  61  963   17    6]
 [  69   14 1194   48]
 [    0    7   86  846]]
```

Accuracy: 91.01%

Classification Report:				precision	recall	f1-score	support
0	0.93	0.92	0.92	0.92	1837		
1	0.90	0.92	0.91	0.91	1047		
2	0.87	0.90	0.89	0.89	1325		
3	0.94	0.90	0.92	0.92	939		
accuracy				0.91	5148		
macro avg	0.91	0.91	0.91	0.91	5148		
weighted avg	0.91	0.91	0.91	0.91	5148		

• Logistic Regression

```
[24]: #Splitting DataSet
xc = data_norm.drop(['Price Class'], axis = 1)
yc = data_norm['Price Class']
xc_train, xc_test, yc_train, yc_test = train_test_split(xc, yc, test_size = 0.3, random_state = 131)
```

```
[25]: #Train Logistic Regression model
logisticmodel = LogisticRegression(max_iter = 20000)
logisticmodel.fit(xc_train, yc_train)
```

```
[25]: LogisticRegression
LogisticRegression(max_iter=20000)
```

```
[26]: #Predict using Logistic model
logpred = logisticmodel.predict(xc_test)
```

```
[27]: #Evaluation of model
print("Accuracy Score: ",accuracy_score(yc_test, logpred) * 100)
print("Classification Report: ",classification_report(yc_test, logpred))
print("Confusion Matrix: ",confusion_matrix(yc_test, logpred))
```

```
Accuracy Score: 98.81507381507382
Classification Report:
precision    recall  f1-score   support

0           0.99      0.99      0.99      1738
1           0.99      0.99      0.99      1339
2           1.00      0.97      0.98       721
3           0.98      0.99      0.98      1350

accuracy          0.99
macro avg          0.99
weighted avg       0.99
```

```
Confusion Matrix: [[1721    0    0   17]
 [  7 1332    0    0]
 [   0   13  697   11]
 [  13    0    0 1337]]
```

Each model was trained and evaluated independently.

6 > Regression Models for Price Prediction

The following regression models were applied:

- Multiple Linear Regression

```
[16]: #Multiple Linear Regression model training
lmodel = LinearRegression()
lmodel.fit(x_train, y_train)
```

```
[16]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
[17]: #Predict Using Multiple Linear Regression
lpred = lmodel.predict(x_test)
```

```
[18]: #EVALUATING Multiple Linear Regression MODEL
lmse = mean_squared_error(y_test,lpred)
print("Mean_squared_Error: ",lmse)
lmae = mean_absolute_error(y_test,lpred)
print("Mean_Absolute_error: ",lmae)
lrmse = np.sqrt(lmse)
print("Root_Mean_Square_Error: ",lrmse)
lscore = r2_score(y_test,lpred)
print("R2_Score: ",lscore)

Mean_squared_Error: 0.0002559293898646446
Mean_Absolute_error: 0.011965602673521194
Root_Mean_Square_Error: 0.015997793281094883
R2_Score: 0.55838130496813
```

- Polynomial Regression

```
[19]: #Using Polynomial Regression
poly = PolynomialFeatures(degree = 5)
x_trainpoly = poly.fit_transform(x_train)
x_testpoly = poly.transform(x_test)
pr = LinearRegression()
pr.fit(x_trainpoly, y_train)
```

```
[19]: ▼ LinearRegression ⓘ ?
LinearRegression()
```

```
[20]: #Predict Using Polynomial Regression
ppred = pr.predict(x_testpoly)
```

```
[21]: #EVALUATING Polynomial Regression MODEL
pmse = mean_squared_error(y_test,ppred)
print("Mean_squared_Error: ",pmse)
pmae = mean_absolute_error(y_test,ppred)
print("Mean_Absolute_error: ",pmae)
prmse = np.sqrt(pmse)
print("Root_Mean_Square_Error: ",prmse)
pscore = r2_score(y_test,ppred)
print("R2_Score: ",pscore)

Mean_squared_Error: 0.00015389542949402731
Mean_Absolute_error: 0.008851345161855706
Root_Mean_Square_Error: 0.012405459664761613
R2_Score: 0.7344459001740062
```

Each model was trained and evaluated independently.

iii. Analysis results

The analysis revealed meaningful patterns between treatment-related features and patient improvement outcomes. Dosage and treatment duration showed noticeable influence on improvement scores. The classification models demonstrated varying performance, with some models achieving higher accuracy and better recall for the positive improvement class.

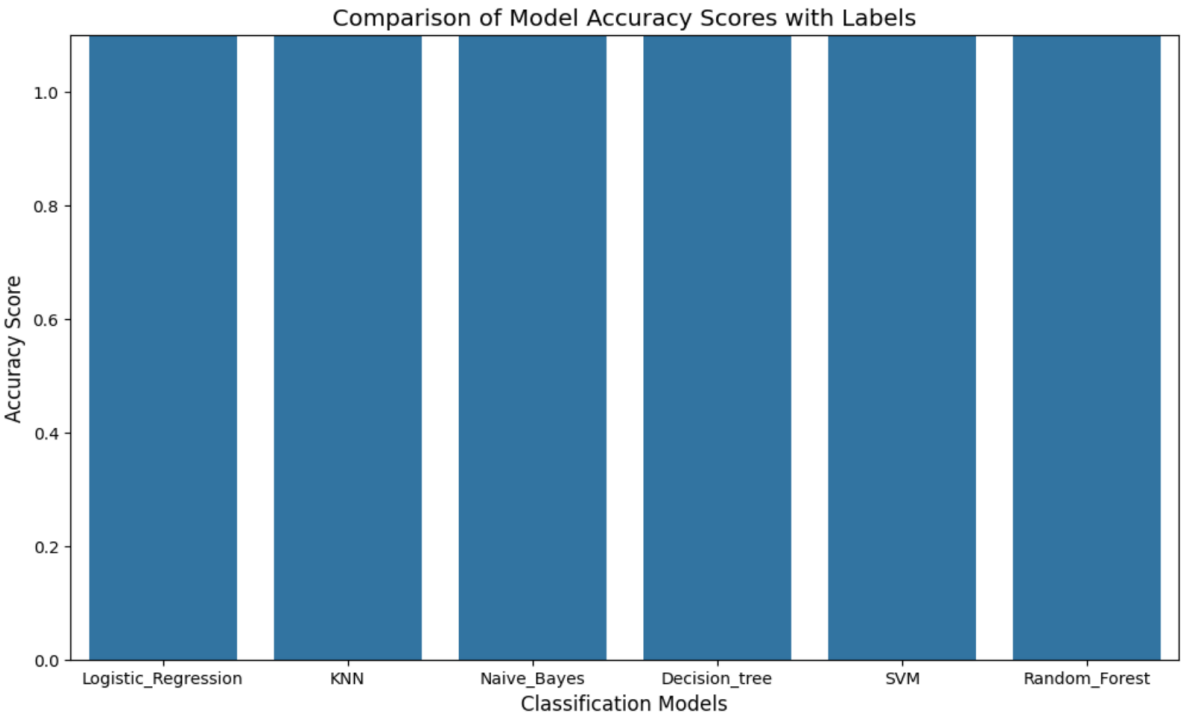
Confusion matrices and classification reports were used to evaluate model performance in terms of accuracy, precision, recall, and F1-score. The comparative analysis helped identify the most effective model for predicting treatment outcomes. The best-fit model was selected based on maximum accuracy and balanced performance across evaluation metrics.

1> Best Classification Model:

```
[64]: #BEST CLASSIFICATION MODEL
model = ['Logistic_Regression', 'KNN', 'Naive_Bayes', 'Decision_tree', 'SVM', 'Random_Forest']
model_accuracies = [lac, kac, nac, dac, sac, rac]
i = np.argmax(model_accuracies)
best_model = model[i]
best_model_accuracy = model_accuracies[i]
print("BEST FIT REGRESSION MODEL : ",best_model)
print("BEST MODEL R2_SCORE : ",best_model_accuracy)

plt.figure(figsize=(12, 7))
ax = sb.barplot(x=model, y=model_accuracies)
plt.xlabel("Classification Models", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)
plt.title("Comparison of Model Accuracy Scores with Labels", fontsize=14)
plt.ylim(0, 1.1)
for container in ax.containers:
    ax.bar_label(container, fmt='%.4f', padding=3, fontsize=10)

BEST FIT REGRESSION MODEL : Logistic_Regression
BEST MODEL R2_SCORE : 99.6891996891997
```

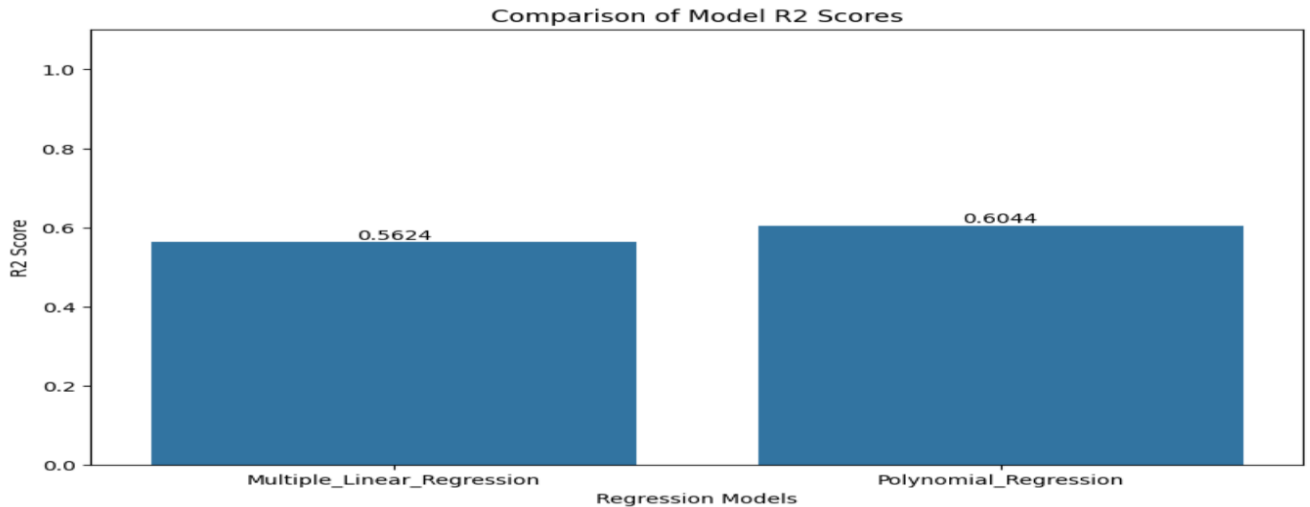


2> Best Regression Model:

```
#Finding the best Regression model
model = ['Multiple_Linear_Regression', 'Polynomial_Regression']
model_r2_scores = [lscore, pscore]
i = np.argmax(model_r2_scores)
best_model = model[i]
best_model_score = model_r2_scores[i]
print("BEST FIT REGRESSION MODEL : ",best_model)
print("BEST MODEL R2_SCORE : ",best_model_score)

# Create the bar plot
plt.figure(figsize=(10, 6))
ax = sb.barplot(x=model, y=model_r2_scores)
plt.xlabel("Regression Models")
plt.ylabel("R2 Score")
plt.title("Comparison of Model R2 Scores")
plt.ylim(0, 1.1)
ax.bar_label(ax.containers[0], fmt='%.4f')

BEST FIT REGRESSION MODEL : Polynomial_Regression
BEST MODEL R2_SCORE : 0.6043793350047384
[Text(0, 0, '0.5624'), Text(0, 0, '0.6044')]
```



iv. Visualization

Visualizations played a critical role in interpreting the dataset and model outcomes. The following visual techniques were used:

- **Count Plots** to analyze the distribution of the target variable
- **Tree Plots** for visualization of Decision tree and Random Forest
- **Correlation Heatmaps** to identify relationships between numerical features
- **Confusion Matrix Heatmaps** with model-specific color palettes to evaluate classifier performance
- **Accuracy Ranking Bar Charts** to compare models and highlight the best-performing classifier

These visualizations enhanced interpretability, supported analytical conclusions, and provided intuitive insights into both data behavior and model effectiveness.

5. Conclusion

This project successfully demonstrated the application of predictive analytics and machine learning techniques to analyze and predict house prices and classify properties into different price categories. Through systematic exploratory data analysis (EDA) and comprehensive preprocessing, the housing dataset was transformed into a structured and model-ready format suitable for supervised learning tasks. A categorical target variable, **Price_Class**, was derived from the house price to enable effective classification of properties.

Multiple machine learning algorithms, including **K-Nearest Neighbors (KNN)**, **Support Vector Machine (SVM)**, **Decision Tree**, and **other regression and classification models**, were implemented and evaluated using standard performance metrics. For regression tasks, metrics such as **R² score**, **Mean Absolute Error (MAE)**, and **Root Mean Squared Error (RMSE)** were used, while classification performance was assessed using **accuracy**, **confusion matrix**, **precision**, **recall**, and **F1-score**. Advanced visualizations, including correlation heatmaps, confusion matrix heatmaps, and model comparison charts, provided valuable insights into feature relationships and overall model performance.

The comparative analysis revealed that different models exhibited varying strengths depending on the nature of the task, and the best-performing model was identified based on accuracy, error minimization, and balanced classification performance. This project emphasizes the importance of effective data preprocessing, feature scaling, and appropriate evaluation metrics in building reliable machine learning models. Overall, the study demonstrates how data-driven approaches can support informed decision-making in the real estate domain and serve as a strong foundation for predictive analytics applications.

6. Future scope

The current project can be further enhanced in several ways. Advanced machine learning models such as **Random Forest**, **Gradient Boosting (XGBoost, LightGBM)**, and **Neural Networks** can be implemented to improve prediction accuracy and model robustness. Hyperparameter tuning techniques such as **Grid Search** and **Random Search** can be applied to optimize model performance further.

The dataset can be expanded by incorporating additional real estate features such as neighborhood crime rates, school ratings, proximity to public transportation, and economic indicators. Addressing class imbalance using resampling techniques like **SMOTE** can improve classification accuracy and reduce bias toward dominant price categories.

In future work, the model can be deployed as a **web-based or mobile application** to provide real-time house price predictions and property classification. Additionally, **Explainable AI (XAI)** techniques such as SHAP and LIME can be used to interpret model predictions, making them more transparent and trustworthy for buyers, sellers, and real estate professionals.

7. References

1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
2. McKinney, W. (2010). *Data structures for statistical computing in Python*. Proceedings of the 9th Python in Science Conference, 51–56.
3. Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90–95.
4. Waskom, M. L. (2021). *Seaborn: Statistical data visualization*. Journal of Open Source Software, 6(60), 3021.
5. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.